

საქართველოს აგრარული უნივერსიტეტი



ს ა კ უ რ ს ო პ რ ო ე ქ ტ ი

3-2

ტემპერატურის მონიტორინგისა და კონტროლის სისტემა

შეასრულა:

საინჟინრო-ტექნოლოგიური ფაკულტეტის

მესამე კურსის სტუდენტმა ნათია კანდელაკმა

2025 წ.

სარჩევი

1. ანოტაცია	3
2. შესავალი	4
3. მთავარი ნაწილი	5
4. ელექტრული სქემა და მისი მუშაობის აღწერა და ანგარიში.....	7
5. კოდის აღწერა.....	10
6. გამოყენებული დეტალების ჩამონათვალი	20
7. გამოყენებული ლიტერატურა/ზმულები ინტერნეტიდან	21

ანოტაცია

უამრავ სფეროში სტაბილური ტემპერატურის შენარჩუნება და მისი კონტროლი გადამწყვეტია ეფექტურობისა და უსაფრთხოებისთვის. პროექტი წარმოადგენს ტემპერატურის კონტროლის სისტემას, რომელიც შექმნილია ტემპერატურის მონიტორინგისა და გაგრილების რეგულირებისთვის. სისტემა ამუშავდება, როდესაც ტემპერატურა მომხმარებლის მიერ წინასწარ განსაზღვრულ ზღვარს გადააჭარბებს, ხოლო როდესაც ამ ზღვარს ჩამოსცდება გამაგრილებელი მოწყობილობა კვლავ გამოირთვება.

არსებობს ტემპერატურის კონტროლის უამრავი საშუალება, თითოეულს გააჩნია სხვადასხვა მუშაობის პრინციპი. მაგალითად, პროპორციული კონტროლერები - იყენებს პროპორციულ რეგულირებას მუდმივი ტემპერატურის შესანარჩუნებლად, ასევე PID ტემპერატურის კონტროლერები - ითვლის სხვაობას სასურველ ტემპერატურულ წერტილსა და მიმდინარე პროცესის ტემპერატურას შორის... ეს არჩევანი დამოკიდებულია ისეთ ფაქტორებზე, როგორიცაა სიზუსტე, რეაგირების დრო, ენერგოეფექტურობა და გამოყენების სირთულე.

კონკრეტული სისტემის მუშაობა დაფუძნებულია თერმორეზისტორზე. მისი მიზანია ეფექტურობისა და სიმარტივის უზრუნველყოფა. მიკროკონტროლერი ამუშავებს სენსორის მონაცემებს და შესაბამისად, აკონტროლებს გაგრილების მოწყობილობას. მომხმარებელს შეუძლია სასურველი ტემპერატურა ორი ღილაკის გამოყენებით მიუთითოს - ერთი ღილაკი ტემპერატურის გასაზრდელად, ხოლო მეორე შესამცირებლად. ორივე ღილაკზე ერთდროულად დაჭერით ხდება ტემპერატურის დაფიქსირება. იმისათვის რომ მომხმარებელმა შეძლოს ტემპერატურის კონტროლი აუცილებელია ინდიკატორი. 7 სეგმენტიანი ინდიკატორი აჩვენებს დაყენებულ ტემპერატურას.

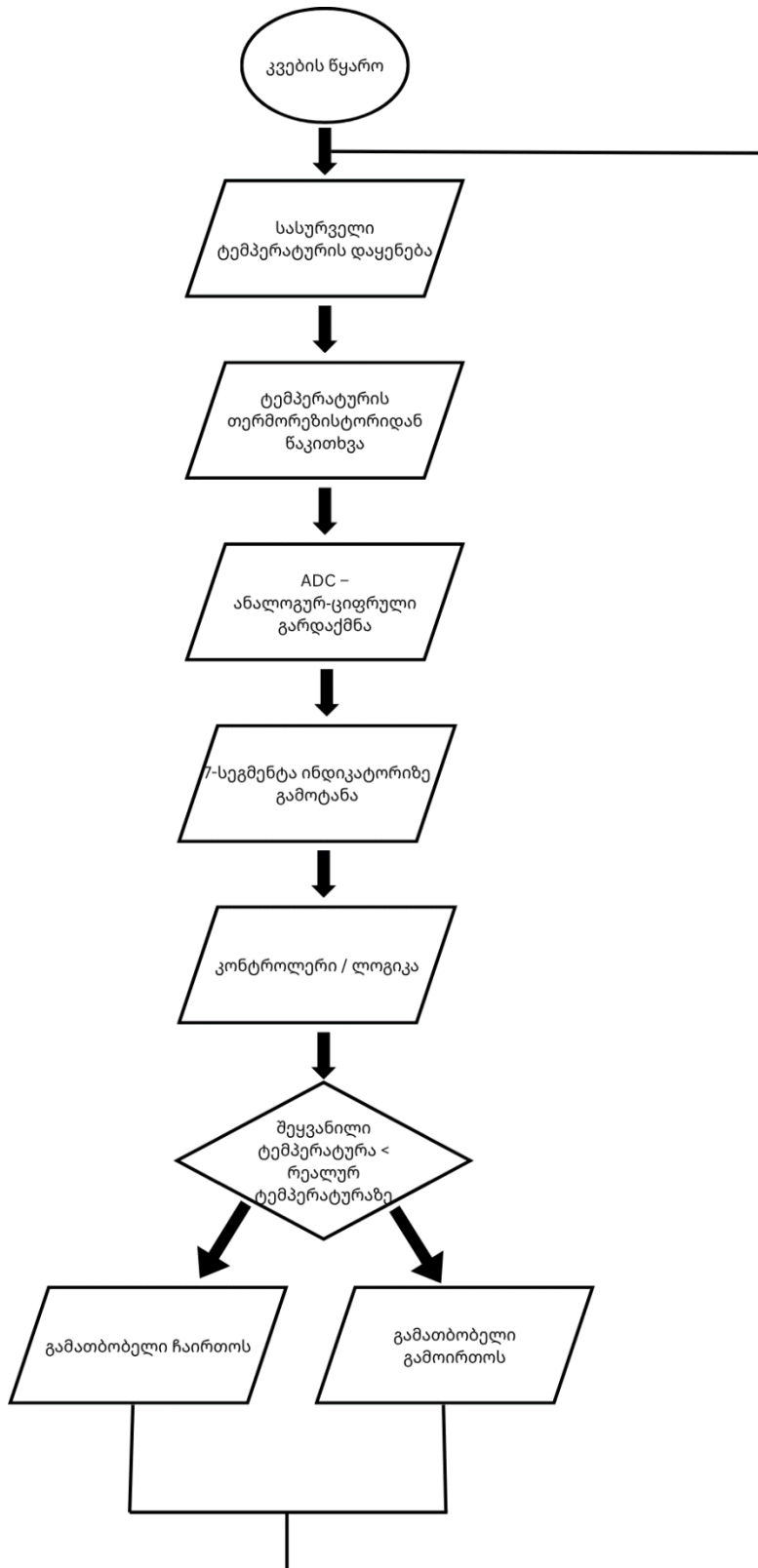
პროექტი შეიძლება გამოყენებულ იქნას ელექტრონული კომპონენტებისა და მოწყობილობების ავტომატური გაგრილების სისტემებში. კლიმატის კონტროლისთვის დახურულ სივრცეებში და სხვა მრავალი აპლიკაციისთვის.

შესავალი

ტემპერატურის მონიტორინგისა და კონტროლის სისტემებს მნიშვნელოვანი როლი აქვს სხვადასხვა ელექტრონულ და მექანიკურ სისტემებში, განსაკუთრებით მაშინ, როდესაც გარემოს პირობების ცვლილება გავლენას ახდენს მოწყობილობის მუშაობაზე. სისტემა შექმნილია იმისათვის, რომ უზრუნველყოს გარემოს ტემპერატურის მუდმივი კონტროლი და საჭიროების შემთხვევაში ჩართოს ან გამორთოს შესაბამისი შემსრულებელი ელემენტები (მაგალითად, გამათბობელი ან გამაგრილებელი). ასეთი სისტემები ფართოდ გამოიყენება მრეწველობაში, აგრარულ სექტორში, სახლის ავტომატიზაციაში, მედიცინაში და სხვა მრავალ სფეროში. გარემოს ტემპერატურის ცვლილება შეიძლება გავლენას ახდენდეს როგორც ადამიანების კომფორტზე, ისე მოწყობილობების მუშაობაზე და სისტემების უსაფრთხოებაზე. სწორედ ამიტომ იქმნება სენსორული სისტემები, რომლებიც უზრუნველყოფენ ტემპერატურის ზუსტ გაზომვას და რეაგირებას მის ცვლილებაზე. ამგვარი სისტემები ხშირად ინტეგრირდება მიკროკონტროლერებთან და სხვა ინტელექტუალურ მოწყობილობებთან, რაც შესაძლებელს ხდის ავტომატურ კონტროლს და ენერგოეფექტურობის გაუმჯობესებას.

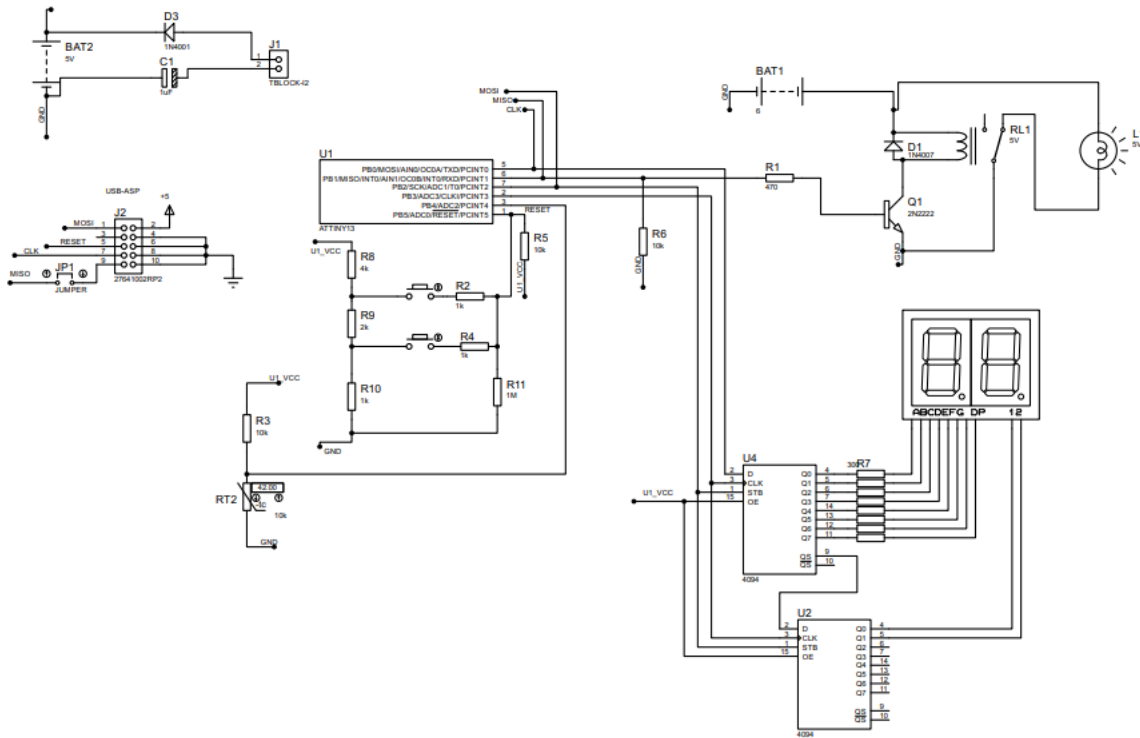
პროექტში გამოყენებულია თერმორეზისტორი, რომლის მეშვეობით ხდება გარემოს ტემპერატურის ანალიზი და შედეგად ხდება მიღებული მნიშვნელობის დაფიქსირება მიკროკონტროლერის მეშვეობით. აღნიშნული პროექტის მიზანია ტემპერატურის კონტროლის პრინციპის პრაქტიკულად განხორციელება, რომელიც უზრუნველყოფს გარემოს ტემპერატურის კონტროლს, ვიზუალურ რეაგირებას და შესაძლო შემსრულებელი ელემენტის მართვას.

მთავარი ნაწილი



წრედი მუშაობს დაახლოებით 5.5 ვოლტზე, რადგან სრულყოფილად იმუშავოს რელემ და ეს არის მიკროკონტროლერისთვის უსაფრთხო დასაშვები ძაბვა. სასურველი ტემპერატურა მომხმარებელს შეყავს გარედან, ორი ღილაკის მეშვეობით. ერთი ღილაკი გამოიყენება ტემპერატურის მოსამატებლად, მეორე შესამცირებლად, ხოლო თუ ორივე ღილაკზე ერთდროული დაჭერა არის დაფიქსირება. ტემპერატურის გასაზომად გამოყენებულია **10k Ω NTC თერმორეზისტორი**, რომლის წინააღმდეგობა ტემპერატურის მატებასთან ერთად იკლებს. ის შეერთებულია როგორც ძაბვის გამყოფი, რაც ანალოგურ სიგნალს წარმოქმნის. მიკროკონტროლერი კითხულობს თერმისტორის ანალოგურ ძაბვას და გარდაქმნის მას ციფრულ მნიშვნელობად ADC სისტემის მეშვეობით. მიღებული მნიშვნელობა წარმოადგენს რეალურ ტემპერატურას. ტემპერატურის მნიშვნელობა გამოისახება 7-სეგმენტიან ინდიკატორზე. მასზე მონაცვლეობით აისახება ჯერ მომხმარებლის მიერ შეყვანილი ტემპერატურა, ხოლო რამდენიმე წამის შემდეგ რეალური ტემპერატურა. მიკროკონტროლერი ადარებს თერმისტორის მიერ განსაზღვრულ რეალურ ტემპერატურას მომხმარებლის მიერ შეყვანილ სასურველ მნიშვნელობას. შედარება ხდება პროგრამულად, წინასწარ განსაზღვრული ლოგიკის მიხედვით. თუ **რეალური ტემპერატურა მაღალია** მომხმარებლის მიერ შეყვანილ ტემპერატურაზე $\rightarrow$ სისტემა აქტივირებს რელეს. თუ ტემპერატურა **სასურველს დაბალია** $\rightarrow$ რელე ითიშება. სისტემა მუშაობს უწყვეტ ციკლში და ახდენს მუდმივ მონიტორინგს.

ელექტრული სქემა და მისი მუშაობის აღწერა



მუშაობის აღწერა:

ტემპერატურის გასაზომად გამოყენებულია 10 კომიანი თერმორეზისტორი NTC, რომლის წინააღმდეგობა ტემპერატურის მატებასთან ერთად იკლებს. ანალოგურ სიგნალის წასაკითხად გამოყენებულია ძაბვის გამყოფი და მისი ცენტრალური წერტილი დაკავშირებულია მიკროკონტროლერის PB4 შესასვლელთან, მიკროკონტროლერი კითხულობს მისგან მიღებულ ანალოგურ სიგნალს და გარდაქმნის მას ციფრულ მნიშვნელობად ADC-ის მეშვეობით.

სქემას აქვს 2 დილაკი, ერთი ტემპერატურის მოსამატებლად, მეორე ტემპერატურის შესამცირებლად, ხოლო ორივე დილაკზე ერთდროული დაჭერით მომხარებელი აფიქსირებს სასურველ ტემპერატურას. ორივე დილაკი დაკავშირებულია რეზისტორების R2, R4, R9 და R10 მეშვეობით მიკროკონტროლერის ერთ ფეხთან PB5 და იმისათვის, რომ მოხდეს მათი წაკითხვა შექმნილია მრავალდონიან ძაბვის გამყოფი. მიკროკონტროლერი განასხვავებს სხვადასხვა დილაკების დაჭერას

(გაზრდას, შემცირებას და ორივეს ერთად) PB5-ზე წაკითხული ძაბვის დონის მიხედვით.

7 სეგმენტა ინდიკატორზე თავდაპირველად აისახება ის რიცხვი, რომელიც გარედან მიეწოდება, ხოლო რამდენიმე წამის შემდეგ აინთება თერმორეზისტორიდან წაკითხული ტემპერატურა და ხდება ამ ტემპერატურების გამოტანის ცვლილება ციკლურად. 7 სეგმენტა ინდიკატორზე გამოდის როგორც მომხმარებლის მიერ შეყვანილი, ისე თერმორეზისტორიდან წაკითხული ტემპერატურა, ამიტომ იმისათვის რომ განვასხვავოთ რომელი ტემპერატურაა გამოსახული ინდიკატორზე, თერმორეზისტორის მიერ წაკითხულ ტემპერატურას ბოლოში აქვს წერტილი, ხოლო მომხმარებლის მიერ დაყენებულ ტემპერატურას არა.

რელეს მუშაობის პრინციპი ასეთია, როდესაც მომხმარებლის მიერ დაფიქსირებული ტემპერატურა ნაკლებია თერმორეზისტორის მიერ აღქმულ ტემპერატურაზე რელე ჩაირთვება, ხოლო როდესაც პირიქით რელე გამოირთვება, ანუ ეს სისტემა წარმოადგენს გაგრილების სისტემას, თუ რელესთან ჩართული იქნება დამატებით გამაგრილებელ მოწყობილობა. თავდაპირველად 2 ციფრიან 7 სეგმენტა ინდიკატორზე ანთია რიცხვი 30, რადგან მომხმარებელს არ მოუწიოს ტემპერატურის დასაყენებლად 1-დან ღილაკებით ცვლილება, ეს ამცირებს ტემპერატურის დაყენების დროს. 7 სეგმენტა ინდიკატორის სამართავად გამოყენებულია ძვრის რეგისტრი (CD4094), მიკროკონტროლერის U1-ის სამი ფეხი PB0, PB2 და PB3 დაკავშირებულია ძვრის რეგისტრთან. მე-5 ფეხი გამოიყენება მონაცემების გადასაცემად რომელიც უკავშირდება ძვრის რეგისტრის მეორე ფეხს, ანუ მიმდევრობით 8 ბიტიან ინფორმაციას გადავცემთ ძვრის რეგისტრის data ფეხს, მიკროკონტროლერის მე-2 ფეხი უკავშირდება ძვრის რეგისტრის მესამე ფეხს, რომელიც წარმოადგენს clock სიგნალის შემავალ ფეხს, ხოლო მიკროკონტროლერის მე-7 ფეხი უკავშირდება ძვრის რეგისტრის პირველ ფეხს რომელიც არის strobe ფეხი, ანუ ამ ფეხის საშუალებით ხდება ინფორმაციის ცვლილების კონტროლი. იმისათვის, რომ მოხდეს ორ ციფრიანი 7 სეგმენტა ინდიკატორის მართვა, საჭიროა ორი ძვრის რეგისტრი. პირველი ძვრის რეგისტრის ინვერსიული გამოსასვლელი დაკავშირებულია მეორე ძვრის რეგისტრის data შესასვლელზე. U4-ის (CD4094) პარალელური გამოსასვლელბი (Q0-Q7) მიერთებულია 7 სეგმენტაზე წინაღობებით რათა არ მოხდეს შუქ დიოდების გადაწვა.

$$R = \frac{V - V_{LED}}{I_{LED}}$$

ყველა შუქდიოდს ფერის მიხედვით აქვს სხვადასხვა ძაბვა. ლურჯი 3.1 ვ.

$$R_{ლურჯი} = \frac{5-3.1}{0.02} = 100 \text{ ომი}$$

სქემაში გამოყენებულია 300 ომი წინაღობები, რადგან თავიდან იქნეს აცილებული ყველანაირი საფრთხე.

ხოლო მეორე ძვრის რეგისტრის Q0 და Q1 გამოსასვლელები გამოყენებულია 7 სეგმენტა ინდიკატორის ციფრების სამართავად. ის გამოიყენება ორ 7-სეგმენტიან ციფრს შორის მულტიპლექსირებისთვის, რაც საშუალებას იძლევა მოხდეს ორნიშნა ინდიკატორის კონტროლი ნაკლები მიკროკონტროლერის პინით.

ტრანზისტორი Q1 (2N2222) რელეს კოჭას გადამრთველის ფუნქციას ასრულებს. მას მიკროკონტროლერი ბაზის რეზისტორის (R1) მეშვეობით მართავს. როდესაც ფაქტობრივი ტემპერატურა აღემატება დადგენილ მნიშვნელობას, მიკროკონტროლერი მაღლა წევს Q1-ის ბაზას, ააქტიურებს რელეს და კვებავს დატვირთვას (გამაგრებელ მოწყობილობას). D1 (1N4007) უზრუნველყოფს უკუგდებისგან დაცვას რელეს კოჭას გასწვრივ.

კოდის აღწერა:

კოდი Attny13 მიკროკონტროლერის გამოყენებით ახდენს ტემპერატურის კონტროლირებადი სისტემის რეალიზებას. ის კითხულობს ტემპერატურის მონაცემებს NTC თერმისტორიდან, საშუალებას აძლევს მომხმარებელს დააყენოს სასურველი ტემპერატურა დაკავშირებული ორი დილაკის გამოყენებით და აჩვენებს როგორც მიმდინარე, ასევე დაყენებულ ტემპერატურას 2-ნიშნა 7-სეგმენტიან ინდიკატორზე ორი CD4094 ძვრის რეგისტრის გამოყენებით. სისტემა რთავს რელეს გაზომილ და დაყენებულ ტემპერატურას შორის შედარების მიხედვით.

```
#define F_CPU 9600000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <stdbool.h>
```

თავდაპირველად გაკეთებულია მიკროკონტროლერის მომუშავე სიხშირის განსაზღვრა, რომელიც ATtiny13-ს შემთხვევაში არის 9.6MHz, და გამოცხადებულია ლოგიკური მონაცემთა ტიპებისა და მნიშვნელობების ბიბლიოთეკა, დაყოვნების ბიბლიოთეკა, წყვეტის ბიბლიოთეკა და I/O რეგისტრების ბიბლიოთეკა.

```
#define RELAY_PIN PB1 // rele
// 4094 shift registri
#define DATA_PIN PB0
#define STB_PIN PB2
#define CLK_PIN PB3
#define BUTTONS_ADC 0 // PB5 / ADC0
#define THERMISTOR_ADC 2 // PB4 / ADC2 thermistor
```

შემდეგ განსაზღვრულია მიკროკონტროლერის პინების სიმბოლური სახელები, რათა კოდი უფრო ადვილად წასაკითხი და შენახული იყოს:

RELAY_PIN, მინიჭებულია PB1-ზე, რომელიც აკონტროლებს რელეს (ვენტილატორი ან სხვა დატვირთვა).

DATA_PIN, STB_PIN, CLK_PIN → ესენი დაკავშირებულია 4094 ძვრის რეგისტრთან და გამოიყენება 7-სეგმენტიანი ინდიკატორის სამართავად;

DATA_PIN (PB0) – მიმდევრობითი მონაცემების შეყვანა.

CLK_PIN (PB3) – clock შეყვანა ცვლის ბიტებისთვის.

STB_PIN (PB2) – strobe მონაცემების გამოსავალზე გადასაცემად.

BUTTONS_ADC დაკავშირებულია ADC0-თან (PB5), რომელიც გამოიყენება ღილაკის მდგომარეობების ანალოგური ძაბვის დონეების მეშვეობით წასაკითხად.

THERMISTOR_ADC დაკავშირებულია ADC2-თან (PB4), რომელიც გამოიყენება NTC თერმისტორიდან ანალოგური მნიშვნელობის წასაკითხად.

```
#define SET(port, pin) (port |= (1 << pin))
#define CLR(port, pin) (port &= ~(1 << pin))
```

ამ მაკროების მიზანია კოდის მკაფიოდ და მარტივად წაკითხვა, რათა არ მოგიწიოს მუდმივად ბიტური ოპერაციების გაწერა ხელით.

SET(port, pin) — აყენებს მითითებულ პინს მაღალი მდგომარეობით (log. "1"). მაგ., SET(PORTB, PB1) ნიშნავს: ჩართე PB1.

CLR(port, pin) — ასრულებს საპირისპიროს: თიშავს პინს (აყენებს ლოგიკურ "0"-ზე).

```
#define DP_BIT 0x80
```

DP_BIT — წარმოადგენს 7-სეგმენტთან დისპლეიზე წერტილის (decimal point) ბიტს. ჩვეულებრივ, ეს არის ყველაზე მაღალი ბიტი (bit 7, ანუ 0x80), რომელსაც ვიყენებთ იმ შემთხვევაში, თუ გვინდა რომ წერტილი ავანთოთ.

```
const uint8_t digits[] = { 0b00111111, // 0
    0b0000110, // 1
    0b01011011, // 2
    0b01001111, // 3
    0b01100110, // 4
    0b01101101, // 5
    0b01111101, // 6
    0b00000111, // 7
    0b01111111, // 8
    0b01101111 }; // 9
```

ეს მასივი განსაზღვრავს სეგმენტების კომბინაციას თითოეული ციფრის გამოსატანად 7-სეგმენტთან ინდიკატორზე. თითოეული ბიტი წარმოადგენს კონკრეტულ სეგმენტს (a–g), და თუ ბიტი ჩართულია (1), შესაბამისი სეგმენტი ინთება.

```
const uint8_t digit_select[2] = {
    0x01, // Right digit (Q0 of 2nd 4094)
    0x02 // Left digit (Q1 of 2nd 4094)
```

```
};
```

ეს მასივი გამოიყენება ორი ციფრიანი 7-სეგმენტიანი ინდიკატორის ციფრების გასაკონტროლებლად. 7-სეგმენტიანი ინდიკატორის თითოეული ციფრი უნდა ჩაირთოს ცალ-ცალკე სწრაფად მონაცვლეობით (მულტიპლექსირება). ამ დროს ერთი ციფრი ჩაირთვება, ხოლო მეორე გაითიშება — და ასე სწრაფად ერთმანეთის მიყოლებით (მომხმარებლისთვის ჩანს, თითქოს ორივე ერთად ანთია). 0x01 და 0x02 ნიშნავს რომ 4094 შიფტ რეგისტრის Q0 და Q1 ბიტებით კონტროლდება ციფრები:

- Q0 = 1 → მარჯვენა ციფრი ჩაირთოს.
- Q1 = 1 → მარცხენა ციფრი ჩაირთოს.

```
const uint16_t temp_thresholds[] =
{569, 546, 523, 500, 478, 455, 434, 413, 392, 373, 353, 335, 317, 300, 284, 269, 254, 240, 227, 214, 202};
```

ეს არის ADC მნიშვნელობების შესაბამისი ტემპერატურული ზღვრები, რომელიც 10კომი NTC თერმისტორის მახასიათებლებს ემთხვევა. temp_thresholds[] არის ADC წაკითხული მნიშვნელობების სია, რომლებიც სხვადასხვა ტემპერატურას შეესაბამება. მასივში 21 მნიშვნელობაა, რაც ნიშნავს რომ ტემპერატურა იზრდება 2°C ნაბიჯით: 20°C-დან 60°C-მდე. მასზე მუშაობს adc_to_temperature() ფუნქცია, რომელიც აფარდებს ADC-ს წაკითხულ მნიშვნელობას ამ ზღვრებზე და აბრუნებს შესაბამის ტემპერატურას.

ამ მნიშვნელობების მისაღებად გამოვიყენე ფორმულები:

$$R = R_0 * e^{B(\frac{1}{T} - \frac{1}{T_0})}$$

$$ADC = \frac{1023 * R_{div}}{R + R_{div}}$$

```
typedef enum {
    BTN_NONE,      //0, gilaki araa dawerili
    BTN_INC,       //1 vumatebt
    BTN_DEC,       //2 vaklebt
    BTN_BOTH       //3 orivezea dawerili
} ButtonState;
```

ეს არის enum ტიპის ცვლადების აღწერა, რომლის მეშვეობით კოდი აღწერს ღილაკების მდგომარეობას. enum — ანუ ჩამონათვალ ტიპი, რომელიც გამოიყენება კოდის წაკითხვის სიმარტივისა და ლოგიკის გასაუმჯობესებლად.

```
volatile uint16_t timer_counter = 0;
volatile uint8_t display_mode = 0;
```

გლობალური ცვლადები Timer-ისთვის და ეკრანის რეჟიმების ცვლადისთვის.

volatile ეუბნება კომპილატორს: ეს ცვლადი შეიძლება შეიცვალოს კოდის გარეშე, მაგალითად წყვეტის (Interrupt) ან სხვა აპარატურული პროცესის მიერ. ნუ გააკეთებ ოპტიმიზაციას მის გარშემო. ეს ცვლადები იცვლება წყვეტის (ISR) შიგნით.

```
ISR(TIM0_OVF_vect, ISR_NAKED) {
    asm volatile("push r16");
    asm volatile("in r16, __SREG__");
    asm volatile("push r16");

    asm volatile("sei"); // Re-enable interrupts if needed
    timer_counter++;
    if (timer_counter >= 185) {
        display_mode = 1 - display_mode;
        timer_counter = 0;
    }

    asm volatile("pop r16");
    asm volatile("out __SREG__, r16");
    asm volatile("pop r16");
    asm volatile("reti");
}
```

ტაიმერის წყვეტა-Timer0 Overflow-ის შემთხვევაში ხდება: timer_counter გაზრდა. ყოველი 5 წამში ეკრანის რეჟიმი იცვლება: set_temp ↔ current_temp.

push რეგისტრებზე (მაგალითად push r16) ნიშნავს რეგისტრის მნიშვნელობის შენახვას stack-ში. როდესაც პროგრამა შედის ISR-ში (interrupt service routine) — ის იყენებს რეგისტრებს. იმისთვის, რომ არ დავკარგოთ ის მნიშვნელობები, რაც მთავარ პროგრამაში გვექონდა ჩაწერილი რეგისტრებში, საჭიროა მათ დროებით შენახვა stack-ში. __SREG__ არის სტატუს რეგისტრი (Status Register) AVR-ზე. შეიცავს ისეთ ინფორმაციას, როგორიცაა Carry, Zero, Negative, Overflow და სხვა დროშები. ISR-ის შიგნით შესაძლოა ამ სტატუსის შეცვლა, რაც დააზიანებდა მთავარ პროგრამას. ამიტომ შევინახეთ __SREG__ stack-ზე. ISR-ის ბოლოს აღვადგენთ ზუსტად იმ მნიშვნელობებს, რომლებიც შევინახეთ: ვაბრუნებთ __SREG__ პირვანდელ მდგომარეობას (ფლაგები იგივე რჩება). ვაბრუნებთ r16 პირვანდელ მნიშვნელობას.

ISR_NAKED ნიშნავს, რომ კომპილატორი არ ჩაწერს ავტომატურად პროლოგსა და ეპილოგს (push/pop).

აქ ხელით ვიცავთ: რეგისტრებს (push r16), სტატუს რეგისტრს (push __SREG__)

```
uint16_t read_adc(uint8_t channel) {
    ADMUX = (ADMUX & 0xF0) | (channel & 0x03); // ATtiny13 აქვს 4 ADC channels (0-3)
    ADCSRA |= (1 << ADSC);
```

```

    while (ADCSRA & (1 << ADSC));
    return ADC;
}

```

მითითებულ ADC არხზე ანალოგური მნიშვნელობის წაკითხვა.

```

ButtonState read_buttons() {
    uint16_t adc = read_adc(BUTTONS_ADC);
    if (adc < 200) return BTN_BOTH;
    if (adc < 300) return BTN_DEC;
    if (adc < 700) return BTN_INC;
    return BTN_NONE;
}

```

ADC მნიშვნელობების მიხედვით ამოიცნობს რომელი ღილაკი (ან ორივე) არის დაჭერილი.

```

uint8_t adc_to_temperature(uint16_t adc) {
    for (uint8_t i = 0; i < 20; i++) {
        uint16_t high = temp_thresholds[i];
        uint16_t low = temp_thresholds[i + 1];

        if (adc >= high)
            return 20 + i * 2;

        if (adc < high && adc >= low) {
            uint8_t offset = ((high - adc) * 2) / (high - low);
            return 20 + i * 2 + offset; //oris bijit maqvs cxrilshi
        }
    }
    return 60;
}

```

NTC 10k თერმისტორის მიხედვით ADC მნიშვნელობიდან ტემპერატურის გამოთვლა.

`uint16_t adc` → ADC-ის ნედლი მნიშვნელობა (0–1023). ვიწყებთ ყველაზე მაღალი ADC მნიშვნელობით (ზედა ზღვარი). `high` — ADC მნიშვნელობა, რაც შეესაბამება $20 + i \cdot 2^{\circ}\text{C}$ -ს. `low` — შემდეგი ზღვარი, ანუ $(20 + (i+1) \cdot 2^{\circ}\text{C})$ -ს.

თუ ADC მნიშვნელობა მეტი ან ტოლია `high`-ზე → ესე იგი ტემპერატურა ქვედა დიაპაზონშია → პირდაპირ ვაბრუნებთ $20 + i \cdot 2^{\circ}\text{C}$.

თუ ADC მნიშვნელობა high-სა და low-ს შორისაა,

$20 + i \cdot 2 + \text{offset}$

ანუ, მივიღეთ ტემპერატურა ორ ზღვარს შორის 2°C ზუსტი ნაბიჯით.

თუ ვერც ერთ დიაპაზონში ვერ მოვხვდით, პროგრამა აბრუნებს 60°C -ს როგორც მაქსიმალურ მნიშვნელობას.

```
void shift_out(uint8_t data) {
    uint8_t i = 8;
    while (i--) {
        (data & 0x80) ? SET(PORTB, DATA_PIN) : CLR(PORTB, DATA_PIN);
        SET(PORTB, CLK_PIN);
        _delay_us(1);
        CLR(PORTB, CLK_PIN);
        data <<= 1;
    }
}
```

მონაცემების გაგზავნა 7-სეგმენტიან დისპლეიზე, ერთ ბაიტს აგზავნის 4094 shift register-ის საშუალებით. $\text{data} \& 0x80 \rightarrow$ იღებს მარცხენა ბიტს (MSB). თუ ბიტი 1-ია $\rightarrow$ DATA_PIN მაღალ მდგომარეობაშია (1), თუ არა 0.

CLK_PIN-ზე $\rightarrow$ მაღალი დონის იმპულსს ვაგზავნით (Clock Pulse), რომ ბიტი ჩაიწეროს რეგისტრში.

$\text{data} \<\<= 1 \rightarrow$ გადავდივართ შემდეგ ბიტზე.

ვიმეორებთ 8-ჯერ $\rightarrow$ მთლიანად 1 ბაიტი გადავგზავნეთ.

```
void display_temperature(uint8_t temp, bool show_dp) {
    uint8_t tens = temp / 10;
    uint8_t units = temp % 10;

    // Tens digit (left)
    shift_out(digit_select[1]);
    shift_out(digits[tens] & ~DP_BIT);
    SET(PORTB, STB_PIN); _delay_us(1); CLR(PORTB, STB_PIN); _delay_ms(5);

    // Units digit (right)
    shift_out(digit_select[0]);
    uint8_t units_pattern = digits[units];
    shift_out(show_dp ? (units_pattern | DP_BIT) : (units_pattern & ~DP_BIT));
    SET(PORTB, STB_PIN); _delay_us(1); CLR(PORTB, STB_PIN); _delay_ms(2);
}
```

}

uint8_t tens = temp / 10; ათეულების ნაწილი - მარცხენა დისპლეი,

uint8_t units = temp % 10; ერთეულების ნაწილი - მარჯვენა დისპლეი,

shift_out(digit_select[1]); ავირჩიეთ მარცხენა ციფრი

shift_out(digits[tens]...) ვაგზავნით შესაბამის კოდს სეგმენტებზე

SET(PORTB, STB_PIN); Latch სიგნალი → 4094 გაანახლებინოს გამოსახვა

_delay_ms(5); დავიცადოთ გამოსახვისთვის

shift_out(digit_select[0]); ავირჩიეთ მარჯვენა ციფრი

shift_out(...units_pattern...) ვაგზავნით ერთეულების შესაბამის კოდს, საჭიროების შემთხვევაში წერტილით (DP)

_delay_ms(2); კიდევ პატარა პაუზა

```
void init_timer() {  
    // viyeneb Timer0  
    // Set prescaler to 1024  
    TCCR0B = (1 << CS02) | (1 << CS00);  
  
    // Enable timer overflow interrupt  
    TIMSK0 = (1 << TOIE0);  
  
    // Enable global interrupts  
    sei();  
}
```

AVR მიკროკონტროლერებს აქვთ სხვადასხვა ტაიმერი, მაგალითად Timer0, Timer1... ATtiny13-ს მხოლოდ 8-ბიტიათი Timer0 აქვს და ამ შემთხვევაში მას ვიყენებთ. ტაიმერის ინიციალიზაცია, Timer0 კონფიგურაცია: Prescaler: 1024. ჩართულია overflow წყვეტა.

ბიტი დროის დაყოფის მნიშვნელობა

CS02 = 1

CS01 = 0

ბიტი დროის დაყოფის მნიშვნელობა
CS00 = 1
შედეგი $9.6 \text{ MHz} \div 1024 = \approx 9375 \text{ Hz}$ (ტაიმერის სიხშირე)

ტაიმერი იტვირთება 9375-ჯერ წამში.

TIMSK0 = (1 << TOIE0);

ეს ნიშნავს, რომ ყოველ ჯერზე, როდესაც ტაიმერი გადაივსება (0xFF → 0x00), გამოიძახება Overflow Interrupt Service Routine (ISR).

გლობალური ინტერპტების ჩართვა sei(); *Set Enable Interrupts* საშუალებას აძლევს მიკროკონტროლერს, რომ რეაგირება მოახდინოს ყველა ჩართულ ინტერპტზე.

```
int main(void) {
    DDRB = 0x0F; // Set PB0-PB3 as outputs
    PORTB = (1 << RELAY_PIN); // Relay off initially

    // Initialize ADC
    ADMUX = 0;
    ADCSRA = (1 << ADEN) |
    (1 << ADPS2) | // Set prescaler to 64
    (1 << ADPS1); // 9.6MHz/64 = 150kHz

    // Initialize timer
    init_timer();

    uint8_t set_temp = 30; //aqedan daiwebs
    uint8_t current_temp;
    ButtonState btn_state = BTN_NONE;
    bool fan_state = 0;
    // uint16_t raw_adc;

    while(1) {
        ButtonState new_btn = read_buttons();
        if (new_btn != btn_state) {
            btn_state = new_btn;

            if (btn_state == BTN_INC && set_temp < 99) set_temp++;
            else if (btn_state == BTN_DEC && set_temp > 0) set_temp--;
        }

        // Read current temperature
        uint16_t raw_adc = read_adc(THERMISTOR_ADC);
        current_temp = adc_to_temperature(raw_adc);

        // releas kontroli
        if (!fan_state && current_temp >= set_temp) {
            CLR(PORTB, RELAY_PIN);
            fan_state = 1;
        } else if (fan_state && current_temp < set_temp) {
            SET(PORTB, RELAY_PIN);
        }
    }
}
```

```

        fan_state = 0;
    }

    display_temperature((display_mode == 0 ? set_temp : current_temp),
display_mode == 1);
    _delay_ms(20);
}
}

```

პორტების ინიციალიზაცია $DDRB = 0x0F$;

$PORTB = (1 \ll RELAY_PIN)$; PB1 (RELAY_PIN) - მაღალ დონეზე (HIGH) , რელე გამორთულია (active-low ლოგიკით მუშაობს).

ADC-ის ინიციალიზაცია

$ADMUX = 0$; // არხი არჩეული არაა ჯერ

$ADCSRA = (1 \ll ADEN) \mid (1 \ll ADPS2) \mid (1 \ll ADPS1)$;

$ADEN \rightarrow$ ADC-ის ჩართვა, პრესკალერი: $9.6\text{MHz} \div 64 \approx 150\text{kHz} \rightarrow$ ADC-სთვის ოპტიმალური სიხშირე.

ტაიმერის ინიციალიზაცია

`init_timer();`

აქედან ტაიმერი იწყებს დათვლას და 5 წამში ერთხელ გამოიძახებს ISR-ს, რომელიც ცვლის `display_mode`-ს (0 ან 1).

პარამეტრების ინიციალიზაცია

`uint8_t set_temp = 30;`

`ButtonState btn_state = BTN_NONE;`

`bool fan_state = 0;`

$set_temp = 30^{\circ}\text{C} \rightarrow$ საწყისი სასურველი ტემპერატურა.

$fan_state \rightarrow$ რელეს მდგომარეობა (0 = გამორთული, 1 = ჩართული).

უსასრულო ციკლი — `while(1)`

დილაკები დაყენებულია **ADC0-ზე (PB5)** $\rightarrow$ წაიკითხება ADC-ს მეშვეობით.

ანალოგური მნიშვნელობების დიაპაზონი $\rightarrow$ კონკრეტული დილაკისთვის.

ლილავზე დაჭერით ტემპერატურა იზრდება ან მცირდება.

```
uint16_t raw_adc = read_adc(THERMISTOR_ADC);
```

```
current_temp = adc_to_temperature(raw_adc);
```

ADC კითხულობს თერმისტორის მიერ მოცემულ ანალოგურ მნიშვნელობას.

ფუნქცია `adc_to_temperature()` → აკონვერტირებს ADC მნიშვნელობას ტემპერატურაში (°C).

```
_delay_ms(20);
```

მცირე პაუზა პროცესორის ზედმეტ დატვირთვას თავიდან აცილებს.

გამოყენებული დეტალების ჩამონათვალი

1. წინაღობა : 1k (9), 1M(1), 10k (3), 300 (9)
2. მიკროკონტროლერი Attiny13 (1)
3. 2ც ღილაკი
4. რელე 5V
5. ტრანზისტორი 2N2222
6. თერმორეზისტორი 10k NTC
7. ძვრის რეგისტრი - CD4094
8. ორმაგი 7 სეგმენტა ინდიკატორი
9. კონდენსატორი 100uF
10. კონექტორი 10 ფეხიანი, 2 ფეხიანი
11. გადამრთველი (1)
12. სამონტაჟო დაფა
13. კვების წყარო

გამოყენებული ლიტერატურა/ზმულები ინტერნეტიდან

1. http://www.herbert-dingfelder.de/?page_id=568
2. [https://students.aiu.edu/submissions/profiles/resources/onlineBook/M4K3T4_Engineering%20Fundamentals_%20An%20Introduction%20to%20Engineering,%20Fourth%20Edition%20\(2010\).pdf](https://students.aiu.edu/submissions/profiles/resources/onlineBook/M4K3T4_Engineering%20Fundamentals_%20An%20Introduction%20to%20Engineering,%20Fourth%20Edition%20(2010).pdf)
3. <https://ww1.microchip.com/downloads/en/DeviceDoc/ATtiny13A-Data-Sheet-DS40002307A.pdf>
4. <https://www.ti.com/lit/ds/symlink/cd4094b.pdf>
5. <https://www.farnell.com/datasheets/95204.pdf>